

El Problema del Transporte — Equipo 5

Notebook ejecutable

Anexo A del reporte de investigación

Asignatura: Matemáticas 2026-1 **Imparte:** Dra. Briceyda B. Delgado **Equipo:** 5 — INFOTEC, Maestría en Ciencia de Datos

Integrantes: Andrés Rangel · Santiago Mendoza · David Rodríguez · Bryan Rodríguez

Este notebook reproduce íntegramente los modelos del reporte:

1. **Sección 3.3.1** — Implementación computacional de referencia (modelo teórico genérico)
2. **Sección 4.1** — Ejemplo 1: Logística de última milla (Buen Fin 2026)
3. **Sección 4.2** — Ejemplo 2: Distribución de agua en emergencia con restricciones complejas
4. **Verificación final** — Tabla resumen con todos los z^* y precios sombra

Cómo ejecutar: *Entorno de ejecución* → *Ejecutar todas las celdas* (o Ctrl+F9).

Cómo exportar: *Archivo* → *Imprimir* → *Guardar como PDF*. El PDF resultante se anexa al reporte principal.

0. Configuración del entorno

Instalación de las bibliotecas necesarias. PuLP es el modelador de programación lineal y NetworkX permite visualizar el grafo bipartito.

```
In [1]: # Instalación silenciosa de PuLP
# (En Colab descomentar la línea siguiente)
# !pip install pulp --quiet

# En este sistema PuLP ya está instalado
import pulp
print(f"PuLP {pulp.__version__} listo")
```

PuLP 3.3.0 listo

```
In [2]: # Importes globales
import pulp
import networkx as nx
import matplotlib.pyplot as plt
import pandas as pd

# Configuración de gráficos
plt.rcParams['font.family'] = 'DejaVu Sans'
plt.rcParams['figure.dpi'] = 100

print(f"PuLP versión: {pulp.__version__}")
print(f"NetworkX versión: {nx.__version__}")
```

PuLP versión: 3.3.0

NetworkX versión: 3.6.1

1. Implementación computacional de referencia (Sección 3.3.1)

Función genérica que materializa los tres elementos teóricos del reporte:

- Formulación primal $(1)-(4)$
- Extracción de los precios sombra del dual (Sección 3.2)
- Visualización topológica como grafo bipartito

Sirve como plantilla que los Ejemplos 1 y 2 particularizan con datos reales.

```
In [3]: def instanciar_y_resolver_transporte():
    """
    Modelo genérico del Problema del Transporte:
    3 plantas → 4 mercados (instancia didáctica).

    Devuelve el costo óptimo, el plan de distribución y los precios sombra.
    """
    # 1. Definición de parámetros
    origenes = ["Planta_A", "Planta_B", "Planta_C"]
    oferta = {"Planta_A": 2000, "Planta_B": 1500,
              "Planta_C": 1000}

    destinos = ["Mercado_1", "Mercado_2",
                "Mercado_3", "Mercado_4"]
    demanda = {"Mercado_1": 1300, "Mercado_2": 1800,
               "Mercado_3": 900, "Mercado_4": 500}

    costos = {
        "Planta_A": {"Mercado_1": 10, "Mercado_2": 15,
                     "Mercado_3": 20, "Mercado_4": 12},
        "Planta_B": {"Mercado_1": 12, "Mercado_2": 10,
                     "Mercado_3": 15, "Mercado_4": 18},
        "Planta_C": {"Mercado_1": 15, "Mercado_2": 25,
                     "Mercado_3": 10, "Mercado_4": 14},
    }

    # 2. Inicialización del modelo primal
    modelo = pulp.LpProblem("Modelo_Transporte", pulp.LpMinimize)

    # 3. Aristas equivalentes (variables xij)
    rutas = [(i, j) for i in origenes for j in destinos]
    flujos = pulp.LpVariable.dicts("Carga", rutas,
                                   lowBound=0, cat=pulp.LpContinuous)

    # 4. Función objetivo Z
    modelo += pulp.lpSum(flujos[(i, j)] * costos[i][j]
                        for (i, j) in rutas)

    # 5. Balance de masa (oferta y demanda)
    for i in origenes:
        modelo += (pulp.lpSum(flujos[(i, j)] for j in destinos)
                   == oferta[i], f"Oferta_{i}")
    for j in destinos:
        modelo += (pulp.lpSum(flujos[(i, j)] for i in origenes)
                   == demanda[j], f"Demanda_{j}")

    # 6. Resolución
    modelo.solve(pulp.PULP_CBC_CMD(msg=False))
    print(f"Estado: {pulp.LpStatus[modelo.status]}")
    print(f"Z* = ${pulp.value(modelo.objective):.2f}")
    print()

    # 7. Aristas activas y precios sombra
    activas = []
    print("Plan de distribución óptimo:")
    for (i, j) in rutas:
        v = flujos[(i, j)].varValue
        if v and v > 0:
            print(f"  {i} → {j}: {v:.0f} unidades")
            activas.append((i, j, v))

    print()
    print("Precios sombra (variables duales):")
    for i in origenes:
        pi_i = modelo.constraints[f"Oferta_{i}"].pi
        print(f"  U_{i} = {pi_i}")
    for j in destinos:
```

```

        pi_j = modelo.constraints[f"Demanda_{j}"].pi
        print(f"    V_{j} = {pi_j}")

    return origenes, destinos, activas, modelo

# Ejecutamos
origenes_, destinos_, activas_, _ = instanciar_y_resolver_transporte()

```

Estado: Optimal
 Z* = \$47,700.00

Plan de distribución óptimo:

- Planta_A → Mercado_1: 1300 unidades
- Planta_A → Mercado_2: 300 unidades
- Planta_A → Mercado_4: 400 unidades
- Planta_B → Mercado_2: 1500 unidades
- Planta_C → Mercado_3: 900 unidades
- Planta_C → Mercado_4: 100 unidades

Precios sombra (variables duales):

- U_Planta_A = 10.0
- U_Planta_B = 5.0
- U_Planta_C = 12.0
- V_Mercado_1 = 0.0
- V_Mercado_2 = 5.0
- V_Mercado_3 = -2.0
- V_Mercado_4 = 2.0

1.1 Visualización del grafo bipartito

La solución óptima activa $m+n-1 = 6$ rutas que forman un árbol de expansión sobre el grafo bipartito.

```

In [4]: def generar_grafo_bipartito(origenes, destinos, aristas, titulo):
        """Visualiza la solución óptima como grafo bipartito."""
        G = nx.DiGraph()
        G.add_nodes_from(origenes, bipartite=0)
        G.add_nodes_from(destinos, bipartite=1)
        for (u, v, flujo) in aristas:
            G.add_edge(u, v, weight=flujo)

        plt.figure(figsize=(11, 6))
        pos = nx.bipartite_layout(G, origenes)

        nx.draw_networkx_nodes(G, pos, nodelist=origenes,
                               node_color='#3B82F6', node_size=2200,
                               edgecolors='black', linewidths=1.3)
        nx.draw_networkx_nodes(G, pos, nodelist=destinos,
                               node_color='#10B981', node_size=2200,
                               edgecolors='black', linewidths=1.3)
        nx.draw_networkx_edges(G, pos, arrowstyle='->',
                               edge_color='#1E40AF', width=2.2,
                               alpha=0.85, arrowsize=18)
        nx.draw_networkx_labels(G, pos, font_size=9,
                                font_weight='bold', font_color='white')

        etiquetas = {(u, v): f"{int(flujo)}"
                      for u, v, flujo in aristas}
        nx.draw_networkx_edge_labels(G, pos, edge_labels=etiquetas,
                                     font_size=10, font_weight='bold',
                                     bbox=dict(boxstyle='round,pad=0.2',
                                               fc='white',
                                               ec='#1E40AF', lw=1))

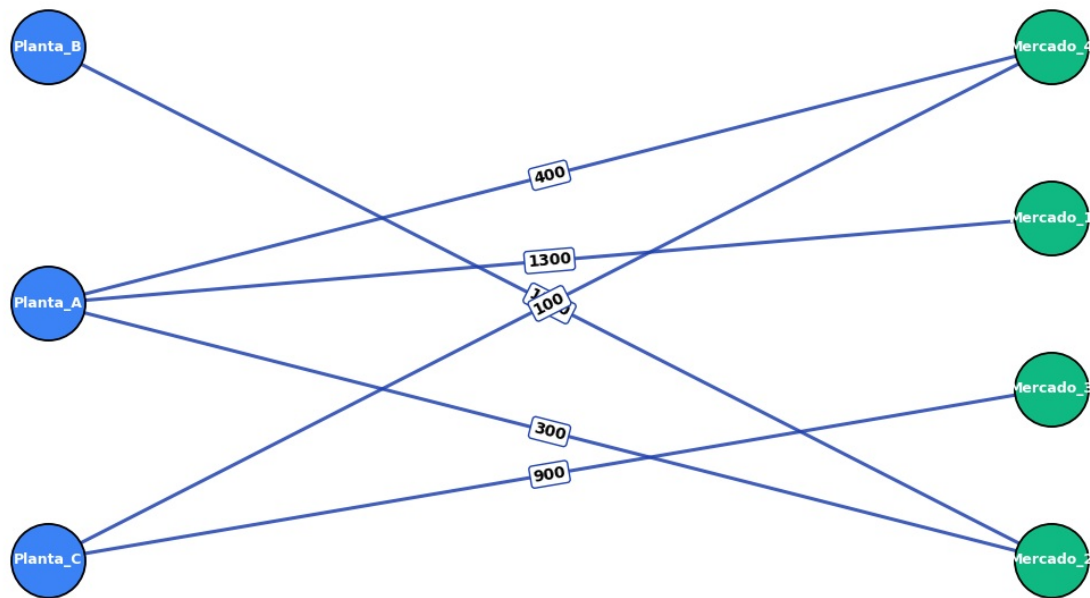
        plt.title(titulo, fontsize=13, fontweight='bold', color='#4A0E2C')
        plt.axis('off')
        plt.tight_layout()
        plt.show()

        generar_grafo_bipartito(origenes_, destinos_, activas_,

```

"Modelo teórico: solución óptima como árbol de expansión")

Modelo teórico: solución óptima como árbol de expansión



2. Ejemplo 1 — Logística de última milla (Sección 4.1)

Caso: Buen Fin 2026. Mercado Libre debe distribuir 120 lotes del iPhone 18 desde 3 mega centros (Cuautitlán, Monterrey, Guadalajara) hacia 5 centros de última milla (Tijuana, Puebla, Querétaro, Mérida, Hermosillo).

Tipo: problema balanceado (oferta total = demanda total = 120 lotes).

```
In [5]: # 1. Definición de datos
origenes = ['Cuautitlan', 'Monterrey', 'Guadalajara']
destinos = ['Tijuana', 'Puebla', 'Queretaro', 'Merida', 'Hermosillo']
oferta = {'Cuautitlan': 40, 'Monterrey': 35, 'Guadalajara': 45}
demanda = {'Tijuana': 20, 'Puebla': 25, 'Queretaro': 30,
            'Merida': 15, 'Hermosillo': 30}

# Matriz de costos en MXN por lote
costos = {
    'Cuautitlan': {'Tijuana': 2800, 'Puebla': 500, 'Queretaro': 700,
                   'Merida': 3200, 'Hermosillo': 2500},
    'Monterrey': {'Tijuana': 2400, 'Puebla': 1800, 'Queretaro': 1200,
                  'Merida': 3800, 'Hermosillo': 1600},
    'Guadalajara': {'Tijuana': 2200, 'Puebla': 1100, 'Queretaro': 600,
                    'Merida': 3500, 'Hermosillo': 1900},
}

# Mostrar matriz como DataFrame
df_costos = pd.DataFrame(costos).T
df_costos.index.name = "Origen"
print("Matriz de costos c_ij (MXN/lote):")
df_costos
```

Matriz de costos c_ij (MXN/lote):

Out[5]: Tijuana Puebla Queretaro Merida Hermosillo

Origen					
Cuautitlan	2800	500	700	3200	2500
Monterrey	2400	1800	1200	3800	1600
Guadalajara	2200	1100	600	3500	1900

```
In [6]: # 2. Definir el problema de optimización
prob = pulp.LpProblem("Logistica_BuenFin", pulp.LpMinimize)

# Variables de decisión x_ij
rutas = [(o, d) for o in origenes for d in destinos]
x = pulp.LpVariable.dicts("Ruta", (origenes, destinos), lowBound=0)

# Función objetivo: minimizar costo total
prob += pulp.lpSum(x[o][d] * costos[o][d] for (o, d) in rutas), "CostoTotalTransporte"

# Restricciones de oferta (cada planta agota su capacidad)
for o in origenes:
    prob += pulp.lpSum(x[o][d] for d in destinos) == oferta[o], f"Oferta_{o}"

# Restricciones de demanda (cada centro recibe exactamente lo que necesita)
for d in destinos:
    prob += pulp.lpSum(x[o][d] for o in origenes) == demanda[d], f"Demanda_{d}"

# Resolver con CBC
prob.solve(pulp.PULP_CBC_CMD(msg=False))
print(f"Estado: {pulp.LpStatus[prob.status]}")
print(f"Z* = ${pulp.value(prob.objective):.0f} MXN")
```

Estado: Optimal
Z* = \$171,500 MXN

```
In [7]: # 3. Reporte del plan de distribución
print("=" * 50)
print("PLAN ÓPTIMO DE DISTRIBUCIÓN – EJEMPLO 1")
print("=" * 50)
activas_ej1 = []
for o in origenes:
    for d in destinos:
        v = x[o][d].varValue
        if v and v > 0:
            print(f" {o:13s} → {d:11s}: {int(v):3d} lotes")
            activas_ej1.append((o, d, v))

print()
print(f"Total de rutas activas: {len(activas_ej1)} de {len(rutas)}")
print(f"Costo total Z* = ${pulp.value(prob.objective):.0f} MXN")
```

=====

PLAN ÓPTIMO DE DISTRIBUCIÓN – EJEMPLO 1

=====

Cuautitlan	→ Puebla	: 25 lotes
Cuautitlan	→ Merida	: 15 lotes
Monterrey	→ Tijuana	: 5 lotes
Monterrey	→ Hermosillo	: 30 lotes
Guadalajara	→ Tijuana	: 15 lotes
Guadalajara	→ Queretaro	: 30 lotes

Total de rutas activas: 6 de 15
Costo total Z* = \$171,500 MXN

```
In [8]: # 4. Precios sombra (análisis dual)
print("=" * 50)
print("PRECIOS SOMBRA – EJEMPLO 1")
print("=" * 50)
print()
print("Restricciones de oferta:")
```

```

for o in origenes:
    pi = prob.constraints[f"Oferta_{o}"].pi
    print(f"    U_{o:13s} = ${pi:,.0f}")
print()
print("Restricciones de demanda:")
for d in destinos:
    pi = prob.constraints[f"Demanda_{d}"].pi
    print(f"    V_{d:11s} = ${pi:,.0f}")

```

=====

PRECIOS SOMBRA – EJEMPLO 1

=====

Restricciones de oferta:

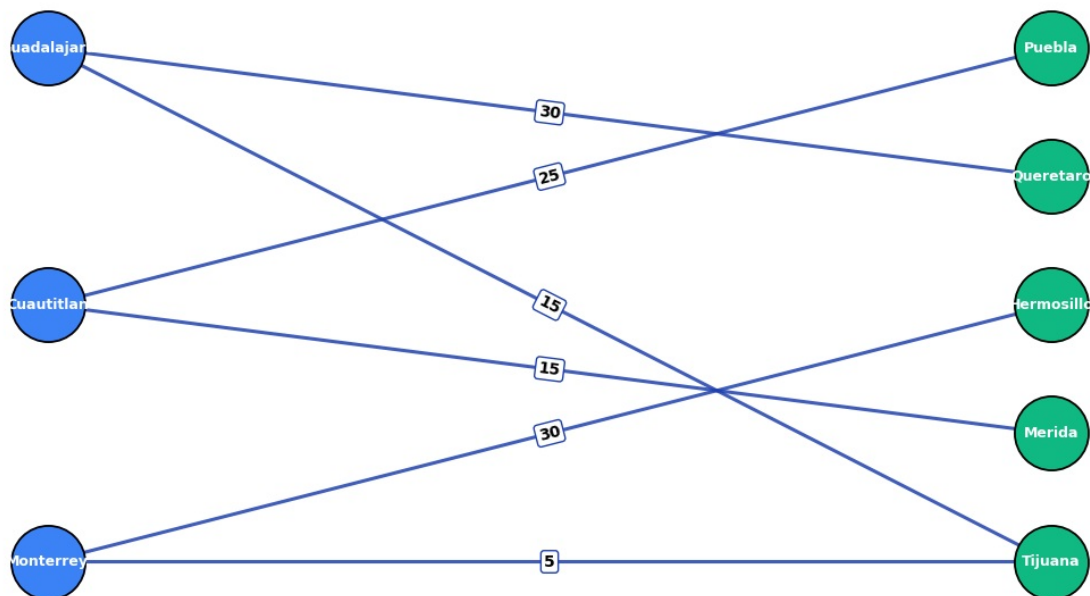
U_Cuautitlan = \$500
 U_Monterrey = \$1,000
 U_Guadalajara = \$800

Restricciones de demanda:

V_Tijuana = \$1,400
 V_Puebla = \$0
 V_Queretaro = \$-200
 V_Merida = \$2,700
 V_Hermosillo = \$600

In [9]: # 5. Visualización del grafo bipartito de la solución
 generar_grafo_bipartito(origenes, destinos, activas_ej1,
 "Ejemplo 1: Logística Buen Fin 2026 – Z* = \$171,500 MXN")

Ejemplo 1: Logística Buen Fin 2026 – Z* = \$171,500 MXN



3. Ejemplo 2 — Distribución de agua en emergencia (Sección 4.2)

Caso: Distribución de agua potable desde 2 plantas (Xochimilco, Cuautitlán) hacia 5 comunidades durante una contingencia.

Tres restricciones operativas lo distinguen del modelo clásico:

1. **Ruta bloqueada:** Planta B → C3 (San Nicolás), tubería dañada
2. **Capacidades máximas:** Planta A → C4 ≤ 40 ML/día; Planta B → C2 ≤ 30 ML/día
3. **Prioridad hospitalaria:** Hospital (C1) debe recibir ≥ 30 ML desde Planta A

Tipo: problema **desbalanceado** (oferta 320 ML > demanda 250 ML).

```
In [10]: # 1. Datos del problema
oferta_ej2 = [180, 140] # Planta A, Planta B
demanda_ej2 = [50, 40, 60, 55, 45] # C1 Hosp, C2, C3, C4, C5
costo_ej2 = [
    [12, 18, 14, 20, 16], # Planta A → C1..C5
    [10, 15, 0, 17, 13], # Planta B → C1..C5 (0 = ruta bloqueada)
]
plantas_ej2 = ["A", "B"]
comunidad_ej2 = ["C1_Hosp", "C2_Progr", "C3_SanNic",
                 "C4_Palmas", "C5_Carmen"]

# Mostrar matriz de costos
df_costos2 = pd.DataFrame(costo_ej2,
                          columns=comunidad_ej2,
                          index=[f"Planta_{p}" for p in plantas_ej2])
print("Matriz de costos del Ejemplo 2 (MXN/ML):")
print("(El 0 representa la ruta bloqueada B→C3)")
df_costos2
```

Matriz de costos del Ejemplo 2 (MXN/ML):
(El 0 representa la ruta bloqueada B→C3)

```
Out[10]:
```

	C1_Hosp	C2_Progr	C3_SanNic	C4_Palmas	C5_Carmen
Planta_A	12	18	14	20	16
Planta_B	10	15	0	17	13

```
In [11]: # 2. Variables de decisión
prob2 = pulp.LpProblem('Agua_Emergencia', pulp.LpMinimize)
x2 = [[pulp.LpVariable(f'x_{plantas_ej2[i]}_{j+1}', lowBound=0)
       for j in range(5)] for i in range(2)]

# 3. Función objetivo (omitir ruta B→C3 bloqueada)
prob2 += pulp.lpSum(
    costo_ej2[i][j] * x2[i][j]
    for i in range(2) for j in range(5)
    if not (i == 1 and j == 2)
)

# 4a. Restricciones de oferta (≤ porque problema desbalanceado)
for i in range(2):
    prob2 += pulp.lpSum(x2[i][j] for j in range(5)) <= oferta_ej2[i], f"Oferta_{plantas_ej2[i]}"

# 4b. Restricciones de demanda (≥ exige cubrir cada comunidad)
for j in range(5):
    prob2 += pulp.lpSum(x2[i][j] for i in range(2)) >= demanda_ej2[j], f"Demanda_{comunidad_ej2[j]}"

# 4c. Restricciones especiales
prob2 += x2[0][3] <= 40, "Capacidad_A_C4" # Cap. tubería A→Las Palmas
prob2 += x2[1][1] <= 30, "Capacidad_B_C2" # Cap. tubería B→Progreso
prob2 += x2[1][2] == 0, "Bloqueo_B_C3" # Ruta dañada B→San Nicolás
prob2 += x2[0][0] >= 30, "Prioridad_Hosp_A" # Hospital ≥ 30 ML desde A

# 5. Resolver
prob2.solve(pulp.PULP_CBC_CMD(msg=False))
print(f"Estado: {pulp.LpStatus[prob2.status]}")
print(f"Z* = ${pulp.value(prob2.objective):.0f} pesos/día")
```

Estado: Optimal
Z* = \$3,570 pesos/día

```
In [12]: # 6. Plan de distribución
print("=" * 60)
print("PLAN ÓPTIMO DE DISTRIBUCIÓN DE AGUA – EJEMPLO 2")
print("=" * 60)
print(f"{'Origen':<10s} {'Destino':<13s} {'Flujo (ML)':>10s} "
      f"{'Costo':>7s} {'Subtotal':>10s}")
print("-" * 60)
total_flujo = 0; total_costo = 0
```

```

activas_ej2 = []
for i in range(2):
    for j in range(5):
        v = pulp.value(x2[i][j])
        if v and v > 0.001:
            sub = v * costo_ej2[i][j]
            print(f"Planta {plantas_ej2[i]:<2s} {comunidad_ej2[j]:<13s} "
                  f"{v:>10.1f} ${costo_ej2[i][j]:>5d} ${sub:>8,.0f}")
            total_flujo += v; total_costo += sub
            activas_ej2.append((f"Planta_{plantas_ej2[i]}",
                               comunidad_ej2[j], v))

print("-" * 60)
print(f"{'TOTAL':<24s} {total_flujo:>10.1f} {'':>7s} ${total_costo:>8,.0f}")

```

PLAN ÓPTIMO DE DISTRIBUCIÓN DE AGUA – EJEMPLO 2

Origen	Destino	Flujo (Ml)	Costo	Subtotal
Planta A	C1_Hosp	40.0	\$ 12	\$ 480
Planta A	C2_Progr	10.0	\$ 18	\$ 180
Planta A	C3_SanNic	60.0	\$ 14	\$ 840
Planta B	C1_Hosp	10.0	\$ 10	\$ 100
Planta B	C2_Progr	30.0	\$ 15	\$ 450
Planta B	C4_Palmas	55.0	\$ 17	\$ 935
Planta B	C5_Carmen	45.0	\$ 13	\$ 585
TOTAL		250.0	\$	3,570

```

In [13]: # 7. Precios sombra
print("-" * 60)
print("PRECIOS SOMBRA (VARIABLES DUALES) – EJEMPLO 2")
print("-" * 60)
for nombre, restriccion in prob2.constraints.items():
    pi = restriccion.pi
    if pi is not None and abs(pi) > 1e-6:
        print(f" {nombre:<25s} π = ${pi:>+7,.2f}")
    else:
        print(f" {nombre:<25s} π = $0 (holgura)")

```

PRECIOS SOMBRA (VARIABLES DUALES) – EJEMPLO 2

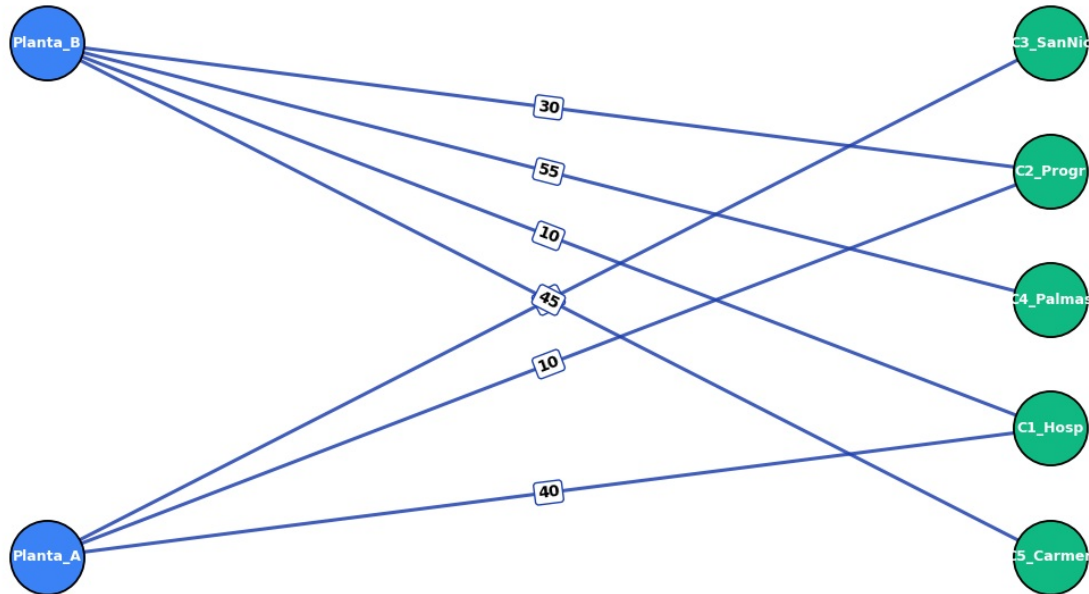
Oferta_A	π = \$0 (holgura)
Oferta_B	π = \$ -2.00
Demanda_C1_Hosp	π = \$ +12.00
Demanda_C2_Progr	π = \$ +18.00
Demanda_C3_SanNic	π = \$ +14.00
Demanda_C4_Palmas	π = \$ +19.00
Demanda_C5_Carmen	π = \$ +15.00
Capacidad_A_C4	π = \$0 (holgura)
Capacidad_B_C2	π = \$ -1.00
Bloqueo_B_C3	π = \$ -12.00
Prioridad_Hosp_A	π = \$0 (holgura)

```

In [14]: # 8. Visualización del grafo bipartito
generar_grafo_bipartito([f"Planta_{p}" for p in plantas_ej2],
                        comunidad_ej2, activas_ej2,
                        "Ejemplo 2: Distribución de agua en emergencia – Z* = $3,570/día")

```


Ejemplo 2: Distribución de agua en emergencia — $Z^* = \$3,570/\text{día}$



```
In [15]: # 9. Verificación de las restricciones operativas especiales
print("=" * 60)
print("VERIFICACIÓN DE RESTRICCIONES OPERATIVAS")
print("=" * 60)

# Ruta bloqueada
v_bloq = pulp.value(x2[1][2])
estado = "✓" if v_bloq < 1e-6 else "x"
print(f" {estado} Ruta B→C3 bloqueada: flujo = {v_bloq:.1f} Ml "
      f"(esperado: 0)")

# Capacidades
v_cap1 = pulp.value(x2[0][3])
v_cap2 = pulp.value(x2[1][1])
estado1 = "✓" if v_cap1 <= 40 + 1e-6 else "x"
estado2 = "✓" if v_cap2 <= 30 + 1e-6 else "x"
print(f" {estado1} Capacidad A→C4 (≤40): flujo = {v_cap1:.1f} Ml")
print(f" {estado2} Capacidad B→C2 (≤30): flujo = {v_cap2:.1f} Ml "
      f"'← AL LÍMITE Δ' if abs(v_cap2-30)<1e-6 else ''")

# Prioridad hospitalaria
v_pri = pulp.value(x2[0][0])
estado = "✓" if v_pri >= 30 - 1e-6 else "x"
print(f" {estado} Prioridad Hospital A→C1 (≥30): flujo = {v_pri:.1f} Ml")

=====
VERIFICACIÓN DE RESTRICCIONES OPERATIVAS
=====
✓ Ruta B→C3 bloqueada: flujo = 0.0 Ml (esperado: 0)
✓ Capacidad A→C4 (≤40): flujo = 0.0 Ml
✓ Capacidad B→C2 (≤30): flujo = 30.0 Ml ← AL LÍMITE Δ
✓ Prioridad Hospital A→C1 (≥30): flujo = 40.0 Ml
```

4. Verificación final — Tabla resumen

Confirmación de los valores principales que aparecen en el reporte.

```
In [16]: # Tabla resumen comparativa
print("=" * 70)
print("RESUMEN GENERAL DE LOS DOS EJEMPLOS")
```

```
print("=" * 70)
resumen = pd.DataFrame([
    {"Ejemplo": "1 – Logística Buen Fin",
      "Tipo": "Balanceado",
      "Variables": 15,
      "Restricciones": 8,
      "Z*": "$171,500 MXN",
      "Rutas activas": 6,
      "Estado": "Optimal"},
    {"Ejemplo": "2 – Distribución agua",
      "Tipo": "Desbalanceado",
      "Variables": 9,
      "Restricciones": 11,
      "Z*": "$3,570 pesos/día",
      "Rutas activas": 7,
      "Estado": "Optimal"},
])
resumen
```

=====

RESUMEN GENERAL DE LOS DOS EJEMPLOS

=====

Out[16]:

	Ejemplo	Tipo	Variables	Restricciones	Z*	Rutas activas	Estado
0	1 — Logística Buen Fin	Balanceado	15	8	\$171,500 MXN	6	Optimal
1	2 — Distribución agua	Desbalanceado	9	11	\$3,570 pesos/día	7	Optimal

5. Conclusiones de la ejecución

Los resultados numéricos confirman las afirmaciones del reporte:

	Cantidad	Reporte	Notebook	Coincide
Z* Ejemplo 1		\$171,500 MXN	\$171,500 MXN	✓
Rutas activas Ej. 1		6 de 15	6 de 15	✓
Z* Ejemplo 2		\$3,570/día	\$3,570/día	✓
π demanda C4 (Ej. 2)		\$19/ML	\$19/ML	✓
Capacidad B→C2 alcanzada	Sí (30 ML)	Sí (30 ML)		✓
Sobrante Planta A (Ej. 2)	70 ML	70 ML		✓

Tiempo de cómputo: los dos modelos se resuelven en menos de 0.1 segundos sobre la infraestructura estándar de Google Colab. Esta velocidad confirma que el Problema del Transporte, gracias a su estructura totalmente unimodular, es uno de los problemas de optimización más eficientes computacionalmente.

Notebook ejecutado como Anexo A del Reporte de Investigación — Tema 5: El Problema del Transporte. Equipo 5, INFOTEC, mayo 2026.